

Exercise 1: Control Structures

Scenario 1: Apply 1% Discount to Customers Above 60 Years

```
DECLARE
    v_age NUMBER;
BEGIN
    FOR rec IN (
        SELECT c.CustomerID, l.LoanID, l.InterestRate, c.DOB
        FROM Customers c
        JOIN Loans l ON c.CustomerID = l.CustomerID
    ) LOOP

        v_age := FLOOR(MONTHS_BETWEEN(SYSDATE, rec.DOB) / 12);

        IF v_age > 60 THEN
            UPDATE Loans
            SET InterestRate = InterestRate - 1
            WHERE LoanID = rec.LoanID;
        END IF;

    END LOOP;

    COMMIT;
END;
/
```

Scenario 2: Set VIP Status for Customers with Balance > 10000

```
ALTER TABLE Customers ADD IsVIP VARCHAR2(5);

BEGIN
    FOR rec IN (SELECT CustomerID, Balance FROM Customers) LOOP

        IF rec.Balance > 10000 THEN
            UPDATE Customers
            SET IsVIP = 'TRUE'
            WHERE CustomerID = rec.CustomerID;
        ELSE
            UPDATE Customers
            SET IsVIP = 'FALSE'
            WHERE CustomerID = rec.CustomerID;
        END IF;

    END LOOP;
```

```
    COMMIT;
END;
/
```

Scenario 3: Send Loan Due Reminders

```
BEGIN
    FOR rec IN (
        SELECT c.Name, l.LoanID, l.EndDate
        FROM Customers c
        JOIN Loans l
        ON c.CustomerID = l.CustomerID
        WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE + 30
    ) LOOP

        DBMS_OUTPUT.PUT_LINE(
            'Reminder: Loan ' || rec.LoanID ||
            ' for customer ' || rec.Name ||
            ' is due on ' || rec.EndDate
        );

    END LOOP;
END;
/
```

Exercise 3: Stored Procedures

Scenario 1: ProcessMonthlyInterest

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType = 'Savings';

    COMMIT;
END;
/
```

Scenario 2: UpdateEmployeeBonus

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
    p_department VARCHAR2,
    p_bonus_percent NUMBER
)
IS
BEGIN
```

```

UPDATE Employees
SET Salary = Salary +
    (Salary * p_bonus_percent / 100)
WHERE Department = p_department;

COMMIT;
END;
/

```

Scenario 3: TransferFunds

```

CREATE OR REPLACE PROCEDURE TransferFunds(
    p_from_account NUMBER,
    p_to_account NUMBER,
    p_amount NUMBER
)
IS
    v_balance NUMBER;
BEGIN

    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = p_from_account;

    IF v_balance < p_amount THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Insufficient Balance'
        );
    END IF;

    UPDATE Accounts
    SET Balance = Balance - p_amount
    WHERE AccountID = p_from_account;

    UPDATE Accounts
    SET Balance = Balance + p_amount
    WHERE AccountID = p_to_account;

    COMMIT;
END;
/

```